

SUPPLY MANAGEMENT SYSTEM

Task Addendum

Organization Master Ledgers

Maps to FSD Addendum 24 | ASP.NET Core 8 | EF Core 8 | SQL Server 2022
Version 1.0 | 2026

FSD source	FSD Addendum 24 - Organization Master Ledgers
Total tasks	5
Total estimated dev days	14 days (~3 working weeks)
Critical priority tasks	2
Total unit test cases	35
Epics	3 epics: financial ledger, product ledger, integration tests
Key architecture	Both master entries written in the SAME transaction as per-supplier/per-product entries. Zero drift by design.

Table of Contents

Table of Contents 2

1. FSD Traceability Summary 3

2. Detailed Task Register 4

3. Sequencing Notes..... 9

1. FSD Traceability Summary

Every task maps to FSD Addendum 24.

Task ID	Title	Days	FSD Ref	Priority
ML-001	MasterFinancialLedger entity and IMasterFinancialLedgerService write integration	3d	Sec 2.1, 6.1	Critical
ML-002	Master Financial Ledger screen and API	3d	Sec 4.1, 5	High
ML-003	MasterProductLedger entity and IMasterProductLedgerService write integration	3d	Sec 3.1, 6.2	Critical
ML-004	Master Product Movement Ledger screen, Product Journey view, and API	3d	Sec 4.2, 5	High
ML-005	End-to-end integration tests and edge case handling	2d	Sec 2.3, 3.2	High
TOTAL	5 tasks	14d		

2. Detailed Task Register

Epic 1 - Master Financial Ledger (FSD Sections 2, 4.1, 6.1)

MasterFinancialLedger entity, the write integration into existing ISupplierLedgerService, all 11 transaction types, and the master ledger screen.

ML-001	Critical	3d	Sec 2.1, 6.1	MasterFinancialLedger entity and IMasterFinancialLedgerService write integration
--------	----------	----	--------------	--

Create the MasterFinancialLedger entity in finance_schema and extend ISupplierLedgerService.PostEntryAsync to write a master entry in the SAME transaction as the per-supplier entry, with a separate organization-level running balance.

Development tasks	Unit test cases
- Add MasterFinancialLedger entity (15 fields per FSD Section 2.1) to FinanceDbContext - EF Core migration: dotnet ef migrations add AddMasterFinancialLedger - Implement IMasterFinancialLedgerService.PostEntryAsync - reads the last master entry's balance_after (UPDLOCK for concurrency), calculates new balance = prev + debit - credit, writes the new entry - Extend ISupplierLedgerService.PostEntryAsync (Addendum 22): after writing the SupplierLedgerEntry, call IMasterFinancialLedgerService.PostEntryAsync within the SAME EF Core transaction (same SaveChangesAsync call) - Denormalise supplier_name into the master entry at write time for reporting without joins - Verify all 11 transaction types from FSD Section 2.2 are wired: INVOICE_APPROVED, PAYMENT, CREDIT_NOTE, DEBIT_NOTE, ADVANCE_PAYMENT, ADVANCE_ADJUSTMENT, RETENTION_HOLD, RETENTION_RELEASE, CHEQUE_BOUNCE_REVERSAL, BAD_DEBT_WRITEOFF, OPENING_BALANCE	- Invoice approval writes both a SupplierLedgerEntry AND a MasterFinancialLedger entry in the same transaction - verified via EF Core transaction log - Master balance correctly accumulates across multiple suppliers: Invoice A (supplier 1, 400,000) + Invoice B (supplier 2, 150,000) = master balance 550,000 - Payment of 300,000 to supplier 1 creates a master credit entry reducing balance from 550,000 to 250,000 - Credit note creates a master credit entry with correct reference - Simulated failure in master ledger write rolls back the per-supplier entry AND the invoice approval - all three are genuinely atomic - Two concurrent invoice approvals (different suppliers) both succeed with correct sequential master balance_after values - All 11 transaction types verified to create the correct debit or credit master entry

Acceptance criteria	- Per-supplier and master entries always written in same transaction - zero drift - Organization-level running balance always accurate under concurrent writes - All 11 event types correctly wired
---------------------	---

ML-002	High	3d	Sec 4.1, 5	Master Financial Ledger screen and API
--------	------	----	------------	--

Build the Finance > Master Payables Ledger screen with chronological entries, running balance, summary cards, supplier filter, drill-down, and export.

Development tasks	Unit test cases
-------------------	-----------------

• GET /api/finance/master-ledger - paginated entries, most recent first, with running balance column. Filters: dateRange, supplierId, transactionType (multi-select), minAmount • GET /api/finance/master-ledger/summary - totalPayables (current balance), totalDebits, totalCredits, netMovement for the filtered period • GET /api/finance/master-ledger/balance - current organization-wide balance (latest balance_after) • React: new screen at Finance > Master Payables Ledger (/finance/master-ledger) • Summary cards at top: Total Payables, Total Debits (period), Total Credits (period), Net Movement • Chronological table with columns: Date, Supplier, Transaction Type, Reference No, Debit, Credit, Balance After, Narration • reference_no is a clickable deep link navigating to the source document (Invoice, Payment, Credit Note, etc.) • Supplier filter dropdown: selecting a supplier filters entries but still shows the org-level running balance; a toggle switches to per-supplier balance view (links to Addendum 22 ledger) • PDF and Excel export via QuestPDF / ClosedXML	• Unfiltered view shows ALL entries across ALL suppliers with correct org-level running balance • Filtering by supplier shows only that supplier's entries but balance_after reflects the org-level running total, not the per-supplier total • Filtering by transaction type = PAYMENT shows only payment entries • Summary cards correctly reflect the filtered period totals • Clicking a reference_no for an invoice navigates to the Invoice detail screen • Excel export contains all visible rows with correct balance column • Toggle to per-supplier view correctly redirects to the Addendum 22 per-supplier ledger
---	---

Acceptance criteria	• Org-level running balance always visible regardless of filters applied • Drill-down to source documents works for all reference types • Export functional in both PDF and Excel
---------------------	---

Epic 2 - Master Product Movement Ledger (FSD Sections 3, 4.2, 6.2)

MasterProductLedger entity, the write integration into existing InventoryLedgerService, source/destination resolution, and the product movement screen.

ML-003	Critical	3d	Sec 3.1, 6.2	MasterProductLedger entity and InventoryLedgerService write integration
--------	----------	----	--------------	---

Create the MasterProductLedger entity in finance_schema and extend InventoryLedgerService.CreateEntryAsync to write a master product entry in the SAME transaction, enriched with source and destination context.

Development tasks	Unit test cases
- Add MasterProductLedger entity (23 fields per FSD Section 3.1) to FinanceDbContext - EF Core migration: dotnet ef migrations add AddMasterProductLedger - Implement IMasterProductLedgerService.PostMovementAsync(context) - accepts a ProductMovementContext containing the product, quantity, transaction type, and resolved source/destination info - Source/destination resolution: map each transaction type to its source and destination - GRN: source=Supplier(name), destination=Warehouse(name); MIV: source=Warehouse(name), destination=Project/Department(name); Return: source=Project(name), destination=Warehouse(name); Transfer: source=Warehouse(from), destination=Warehouse(to); etc. - Extend InventoryLedgerService.CreateEntryAsync (Addendum 20): after writing the InventoryLedgerEntry, call IMasterProductLedgerService.PostMovementAsync within the SAME transaction - Denormalise product_name, product_code, category_name, source_name, destination_name at write time - Calculate total_value = quantity x unit_cost at write time	- GRN receipt of 100 laptops from Lenovo Distributors creates a master product entry with source_type=SUPPLIER, source_name=Lenovo Distributors, destination_type=WAREHOUSE, destination_name=Main Warehouse - Material Issue of 20 to Project A creates entry with source_type=WAREHOUSE, source_name=Main WH, destination_type=PROJECT, destination_name=Tower Block A - Material Return of 10 creates entry with source_type=PROJECT, destination_type=WAREHOUSE - Both InventoryLedgerEntry and MasterProductLedger entry exist in the same transaction - verified via transaction log - Simulated failure in master product write rolls back the inventory ledger entry AND the business action - total_value correctly calculated as quantity x unit_cost - All 10 transaction types from FSD Section 3.2 correctly map source and destination

Acceptance criteria	- Source/destination resolution correct for all movement types - Product and master entries always in same transaction - Denormalised fields populated correctly at write time
---------------------	--

ML-004	High	3d	Sec 4.2, 5	Master Product Movement Ledger screen, Product Journey view, and API
--------	------	----	------------	--

Build the Inventory > Master Product Ledger screen with source/destination columns, summary cards, filters, the Product Journey drill-down, and export.

Development tasks	Unit test cases
GET /api/inventory/master-product-ledger - paginated entries, most recent first. Filters:	Unfiltered view shows all product movements across all warehouses

dateRange, productId, categoryId, warehouseId, transactionType, sourceType, destinationType • GET /api/inventory/master-product-ledger/summary - totalReceiptsQty, totalIssuesQty, netMovement, totalValueMoved for filtered period • GET /api/inventory/master-product-ledger/product-journey/{productId} - complete movement history for one product across all warehouses, chronological • React: new screen at Inventory > Master Product Ledger (/inventory/master-product-ledger) • Summary cards: Total Receipts (qty in), Total Issues (qty out), Net Movement, Total Value Moved • Table columns: Date, Product, Category, Transaction Type, Qty In, Qty Out, Unit Cost, Total Value, Source (type + name), Destination (type + name), Reference No, Warehouse • reference_no clickable: navigates to GRN/MIV/Return/etc. product_name clickable: navigates to Product detail with current stock • Product Journey view: selecting a product and clicking View Journey opens a chronological timeline of every movement for that product - from initial GRN receipt through all issues, returns, transfers, wastage • Filters for source_type and destination_type allow queries like: show me all movements where destination_type = PROJECT (everything that went to projects) • PDF and Excel export	• Filtering by product shows only that product's movements • Filtering by destination_type = PROJECT shows all material issues to projects • Product Journey for a laptop serial shows: GRN receipt from Lenovo, then MIV issue to IT Department - complete lifecycle • Summary cards correctly total qty_in and qty_out for the filtered period • reference_no deep link navigates to the correct source document • Product Journey correctly handles products with movements across multiple warehouses
--	---

Acceptance criteria	• Source/destination columns provide immediate visibility into where products came from and went • Product Journey gives a complete lifecycle view per product • All filters work independently and in combination
-----------------------------------	--

Epic 3 - Worked Example Verification and Edge Cases (FSD Sections 2.3, 3.2)

End-to-end integration tests reproducing the FSD worked examples exactly, plus edge case handling.

ML-005	High	2d	Sec 2.3, 3.2	End-to-end integration tests and edge case handling
--------	------	----	--------------	---

Build integration tests reproducing the exact worked examples from FSD Sections 2.3 and 3.2, plus handling for edge cases: opening balance import, bad debt write-off, and concurrent multi-supplier activity.

Development tasks	Unit test cases
- Integration test: reproduce FSD Section 2.3 financial worked example exactly - 8 entries ending at balance 350,000 with the advance adjustment logic verified - Integration test: reproduce FSD Section 3.2 product movement worked example - 10 entries with correct source/destination for each - POST /api/finance/master-ledger/opening-balance - one-time import endpoint for go-live migration, creates OPENING_BALANCE entries for each supplier with a pre-system payable, protected by a system flag (can only be run once) - POST /api/finance/master-ledger/write-off - bad debt write-off: creates a credit entry removing an amount from the payable balance, requires Finance Manager approval before posting - Handle edge case: cheque bounce reversal creates a debit entry in the master ledger that correctly reverses the earlier credit, restoring the balance to its pre-payment state - Handle edge case: advance payment creates a negative master balance (org in credit position with supplier) - verify this is correctly displayed and does not break balance calculations - Verify concurrent activity: two approvals for different suppliers processed simultaneously produce correct sequential balance_after values with no gap or collision	- Financial worked example: after all 8 entries, master balance_after = 350,000 (not 150,000 - the advance adjustment adds back the 200,000 advance credit) - Product worked example: GRN receipt correctly maps source=Supplier, destination=Warehouse; MIV correctly maps source=Warehouse, destination=Project - Opening balance import for 3 suppliers creates 3 master entries with correct individual debit amounts and a cumulative balance - Bad debt write-off of 15,000 creates a credit entry reducing balance by 15,000, with Finance Manager approval verified - Cheque bounce reversal after a 300,000 payment creates a 300,000 debit entry, restoring the balance to pre-payment - Negative master balance (advance exceeds invoices) correctly displayed as negative, not zero or error - Two concurrent approvals: both succeed, balance_after values are sequential with no gap

Acceptance criteria	- Both FSD worked examples reproduced exactly in integration tests - Opening balance import works correctly for go-live - Edge cases (bounce, write-off, negative balance, concurrency) all handled without errors
----------------------------	--

3. Sequencing Notes

Part A (Financial) and Part B (Product) can be developed in parallel since they extend different existing services.

Dependency	Reason
Addendum 22 (SFM-001) must be deployed before ML-001	ML-001 extends ISupplierLedgerService.PostEntryAsync which must exist first.
ML-001 before ML-002	The screen reads from the MasterFinancialLedger table created by ML-001.
Addendum 20 (IInventoryLedgerService) must exist before ML-003	ML-003 extends CreateEntryAsync which must exist first.
ML-003 before ML-004	The screen reads from the MasterProductLedger table created by ML-003.
ML-001 and ML-003 before ML-005	Integration tests verify both master ledgers are written correctly.
ML-001 and ML-003 can be parallelised	They extend different services (financial vs inventory) with no cross-dependency.

- End of Document -